

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Documentation Technique GBZ

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

SOMMAIRE

1.Introduction - 3 -

2.Technologies Choisies - 4 -

 2.1.Backend - 4 -

 2.2.Admin - 5 -

 2.3.Mobile - 5 -

 2.4 Autres - 6 -

3.Architecture Générale..... - 6 -

 3.1.Vue d’ensemble - 6 -

 3.2.Schéma d’architecture - 7 -

4.Arborescence du Projet - 8 -

 4.1.Arborescence globale - 8 -

 4.2.Organisation des modules - 9 -

5.Installation et Lancement - 10 -

 5.1.Prérequis - 10 -

 5.2.Script de démarrage - 10 -

 5.3.Configuration. Env..... - 11 -

 5.4.Lancement Docker..... - 11 -

 5.5.Lancement Projet..... - 12 -

6.Backend..... - 12 -

 6.1.Structure - 13 -

 6.2.Base de données..... - 14 -

 6.3.Sécurité - 14 -

 6.4.Liste Endpoints - 15 -

7.Admin - 17 -

 7.1.Structure du projet - 18 -

 7.2.Gestion des utilisateurs..... - 19 -

 7.3.Connexion à l’API - 19 -

8.Mobile - 20 -

 8.1.Navigation - 21 -

 8.2.Ecrans et Fonctionnalités - 21 -

9.Développement et Organisation..... - 23 -

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

9.1.Méthodologie	- 23 -
9.2.Gestion GIT	- 23 -
10.Déploiement	- 24 -
10.1.Local	- 24 -
10.2.Déploiement en Ligne.....	- 24 -
11.Conclusion.....	- 24 -
11.1.Bilan personnel.....	- 24 -
11.2.Points d’amélioration	- 24 -

1. Introduction

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Le projet GBZ est une application fullstack réalisée dans le cadre du projet fil rouge de ma formation à Aix Ynov Campus, spécialisation Développeur Web. L'objectif principal était de mettre en pratique l'ensemble des compétences acquises tout au long de l'année à travers le développement complet de l'application.

Plutôt que d'opter pour l'un des sujets proposés, j'ai préféré m'investir dans un projet personnel qui me tenait particulièrement à cœur : concevoir une application mobile dédiée à la musculation et au suivi d'entraînements, accompagnée d'une interface web d'administration permettant la gestion des utilisateurs, des exercices et des séances. Lorsqu'on m'a présenté ce projet, je l'ai tout de suite perçu comme une occasion idéale pour concrétiser une idée que j'avais depuis longtemps.

L'architecture du projet est structurée de manière simple et efficace, reposant sur trois éléments principaux :

- Un backend,
- Une interface d'administration,
- Une application mobile.

Le choix des technologies était totalement libre, ce qui m'a permis d'utiliser des outils et des frameworks modernes, que je voulais aussi utiliser depuis longtemps.

2. Technologies Choisies

Ce projet repose sur une stack technique moderne, pensée pour garantir à la fois performance, clarté et évolutivité. Le choix des technologies s'est fait en fonction de mes préférences personnelles, mais aussi de mon envie de tester certains outils découverts au fil de mes recherches, notamment sur GitHub. Cette combinaison me semblait pertinente pour construire une base solide, facile à maintenir et à faire évoluer dans le temps.

2.1. Backend

Pour la partie backend, le choix s'est porté sur le framework NestJS, avec lequel j'avais déjà eu l'occasion de travailler auparavant. Ce framework se distingue par sa structure claire, sa scalabilité, ainsi qu'une documentation particulièrement bien fournie. Il facilite grandement la mise en place d'une API REST sécurisée et bien organisée.

Côté base de données, Prisma a été retenu comme ORM. C'est un outil que j'avais envie d'explorer depuis un moment, notamment pour sa gestion typée des modèles, sa compatibilité avec PostgreSQL, et sa capacité à simplifier les migrations. Son approche moderne permet une bonne lisibilité du schéma et rend l'interaction avec la base plus intuitive.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Avec le recul, j'ai vraiment apprécié travailler avec NestJS. Ce framework m'a permis d'organiser mon code de façon claire et structurée, ce qui a rendu le développement backend beaucoup plus fluide.

Quant à Prisma, ça a été une vraie découverte. Au départ, la prise en main a été un peu complexe : entre les commandes à retenir et les erreurs rencontrées, la courbe d'apprentissage m'a semblé assez raide. Mais au fil du projet, j'ai fini par bien me familiariser avec l'outil, et aujourd'hui, je le considère presque comme indispensable dans un projet Node.js. La clarté des schémas, la facilité des migrations et les typages automatiques m'ont clairement conquis.

2.2.Admin

Pour ce qui est de l'interface d'administration, j'ai choisi d'utiliser une technologie encore peu connue, mais que j'avais repérée sur GitHub depuis un moment et qui m'intriguait beaucoup. Il s'agit de Refine, un framework open-source basé sur React, spécialement conçu pour construire rapidement des dashboards performants. Refine propose une intégration native avec des outils comme React Query, React Router et Ant Design, ce qui m'a permis de gagner en productivité tout en gardant une grande flexibilité.

Avec le recul, j'ai vraiment apprécié travailler avec cette technologie. Cependant, je la trouve encore un peu jeune : j'ai rencontré pas mal d'erreurs liées aux différents packages, et la documentation associée n'est pas toujours très fournie. Malgré cela, Refine m'a permis de mettre en place un dashboard à la fois utile et performant.

2.3.Mobile

L'application mobile a été développée en React Native, en s'appuyant sur Expo Go, dans le but de simplifier le cycle de développement et de test sur les appareils mobiles. Ce choix technologique m'a permis de construire rapidement une application native, compatible à la fois avec iOS et Android, tout en maintenant une seule base de code.

L'utilisation d'Expo Go a facilité les premiers déploiements, notamment grâce au rechargement en temps réel et à l'absence de configuration native complexe. Cependant, avec le recul, j'ai rencontré plusieurs limitations liées à cette solution. Certaines dépendances spécifiques étaient difficiles à intégrer, et l'accès à des fonctionnalités natives avancées s'est parfois révélé contraignant. Cela a freiné le développement sur certains aspects et a généré une certaine frustration à mesure que le projet gagnait en complexité.

Malgré ces limites, Expo m'a permis de structurer efficacement mon application mobile.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

2.4 Autres

Docker & Docker Compose : J'ai opté pour Docker afin d'isoler chaque service de l'application (backend, admin, base de données). Grâce à Docker Compose, j'ai pu automatiser le lancement complet du projet en local, ce qui m'a permis de gagner beaucoup de temps et d'éviter de nombreux problèmes de configuration entre les différents environnements.

Git & GitHub : Le contrôle de version a été géré avec Git, et tout le code est hébergé sur GitHub. Cela m'a permis de garder un historique clair de l'évolution du projet et de revenir facilement en arrière si nécessaire. J'ai également appliqué la règle des *conventional commits* tout au long du développement pour assurer une meilleure lisibilité et organisation des commits.

GitHub Projects : Pour organiser le travail, j'ai mis en place un tableau Kanban avec GitHub Projects. Je l'ai utilisé pour planifier mes tâches, suivre l'avancement du développement et garder une vision claire des priorités. J'ai également créé une liste de labels personnalisés (backend, frontend, etc.) pour mieux catégoriser les issues. Cet outil a été très utile pour maintenir la structure et la cohérence du projet.

3. Architecture Générale

3.1. Vue d'ensemble

Le projet GBZ est structuré selon une architecture fullstack classique mais modulaire, composée de trois couches principales :

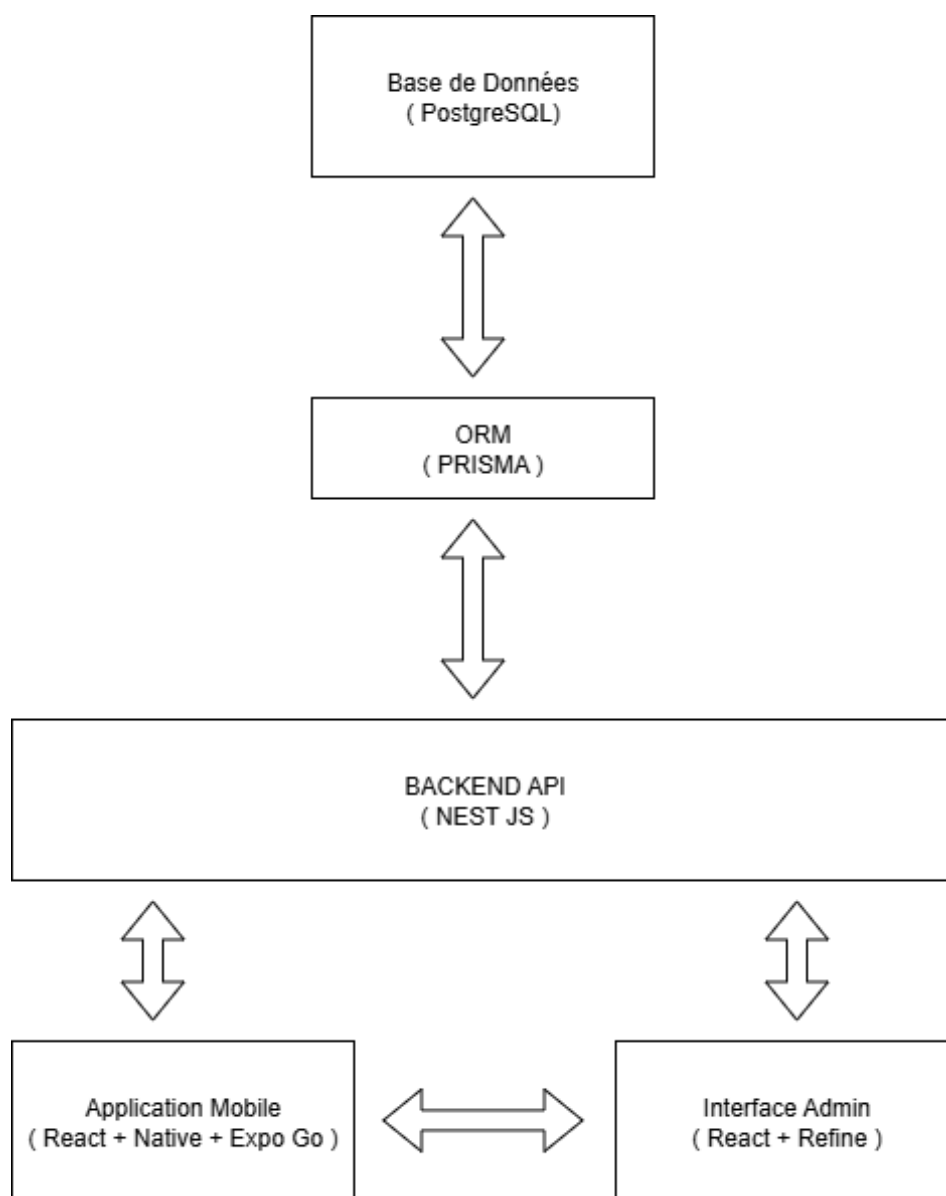
- **Backend** : Il constitue le cœur de l'application, assurant la gestion des données, la logique métier et l'exposition d'une API REST sécurisée. Le backend est développé avec NestJS, ce qui permet une organisation claire du code en modules et services, favorisant la maintenabilité et l'évolutivité.
- **Interface d'administration (Admin)** : Une application web dédiée à la gestion du contenu, des utilisateurs, des séances et des exercices. Cette interface est conçue avec Refine, un framework React spécialisé dans la création rapide de dashboards performants et flexibles.
- **Application mobile** : Destinée aux utilisateurs finaux, cette application React Native (avec Expo Go) permet aux passionnés de musculation de suivre leurs entraînements et de gérer leurs séances directement depuis leur smartphone, sur iOS comme sur Android.

Chaque composant communique principalement via des API REST. L'architecture est pensée pour être découplée et facilement maintenable, facilitant ainsi les évolutions futures. L'utilisation de Docker garantit une isolation des environnements et une portabilité optimale lors du développement et du déploiement.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

3.2. Schéma d'architecture

Voici un schéma représentant l'architecture globale du projet GBZ :



Le projet GBZ est structuré autour de trois composants principaux :

1. **Application Mobile (React Native + Expo Go)**

Cette application est destinée aux utilisateurs finaux, les passionnés de musculation.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Elle communique en **bidirectionnel** avec le backend, envoyant des requêtes (ex : récupération des séances, envoi des données d'entraînement) et recevant des réponses (ex : mise à jour du profil, listes d'exercices).

2. Interface Admin (React + Refine)

Cette interface web permet aux administrateurs et gestionnaires du projet de gérer les utilisateurs, exercices, et séances.

Comme l'application mobile, elle échange en **bidirectionnel** avec le backend pour modifier et consulter les données.

3. Backend (NestJS + Prisma)

Le backend est le cœur métier du projet. Il expose une API REST sécurisée et gère la logique applicative.

Il utilise **Prisma** comme ORM pour interagir efficacement avec la base de données PostgreSQL. Prisma facilite la gestion des données, les migrations, et la cohérence du modèle.

Le backend reçoit et traite les requêtes provenant à la fois de l'application mobile et de l'interface admin.

4. Base de données PostgreSQL

Toutes les données structurées du projet (utilisateurs, exercices, séances, etc.) sont stockées dans cette base relationnelle.

Prisma assure la liaison entre le backend et cette base, garantissant un accès optimisé et sécurisé aux données.

5. Docker Compose

L'ensemble de ces services (backend, base de données, interface admin) est orchestré via Docker Compose.

Cela permet de lancer facilement tout le projet en local dans des conteneurs isolés, assurant une cohérence entre les environnements de développement.

4. Arborescence du Projet

4.1. Arborescence globale

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

L'organisation des fichiers et dossiers du projet GBZ est pensée pour séparer clairement les différentes parties de l'application, facilitant ainsi la maintenance et le développement. Voici la structure globale du projet :

```
GBZ/
├── backend/
├── auth-antd
├── gbz-mobile-app/
├── docs
├── docker-compose.yml
├── start-mobile.sh
├── create-env.sh
├── .
└── README.
```

Chaque dossier correspond à un service ou un aspect spécifique du projet, permettant ainsi de travailler de manière isolée sur chaque partie tout en gardant une vision globale claire.

4.2. Organisation des modules

Backend (backend/)

- Contient le code source de l'API REST développée avec NestJS.
- La logique métier est organisée en modules fonctionnels (ex : utilisateurs, exercices, séances).
- Intègre Prisma pour la gestion des accès à la base de données PostgreSQL.
- Comprend également les fichiers de configuration essentiels (ex : .env, Dockerfile).

Interface d'administration (auth-antd/)

- Projet React utilisant le framework Refine, conçu pour construire rapidement un dashboard performant.
- Organisé autour des composants, pages et services dédiés à la gestion des données.
- Intègre des bibliothèques tierces telles qu'Ant Design pour les composants d'interface utilisateur.

Application mobile (gbz-mobile-app/)

- Application mobile développée en React Native avec Expo Go.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

- Structure en composants, écrans et services API pour assurer la communication avec le backend.
- Contient les fichiers de configuration nécessaires au lancement, tels que .env et app.json.

Documentation (docs/)

- Regroupe la documentation technique détaillant l'architecture, les choix technologiques et les guides développeurs.
- Contient également la documentation fonctionnelle, incluant le cahier des charges et les cas d'utilisation.

Autres

- Script shell (.sh) nécessaire à la configuration et au lancement du projet mobile.
- Fichier Docker Compose pour orchestrer le démarrage de tous les services.
- Fichier .gitignore pour exclure du versioning Git les fichiers temporaires, dépendances et données sensibles.

5. Installation et Lancement

5.1. Prérequis

Avant de commencer, veuillez vous assurer d'avoir installé sur votre machine les outils suivants :

- **Node.js** (version 18 ou supérieure recommandée)
- **Docker** et **Docker Compose** pour gérer les services en conteneurs
- **Expo Go** sur votre téléphone (disponible sur iOS et Android)
- **Git** pour cloner le dépôt

Ces outils sont indispensables pour lancer et faire fonctionner l'ensemble du projet dans les meilleures conditions.

5.2. Script de démarrage

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Pour faciliter la configuration, deux scripts sont mis à votre disposition pour automatiser certaines étapes :

- Le script `create-env.sh` permet de créer automatiquement le fichier `.env` nécessaire à la configuration de la base de données.
- Le script `start-mobile.sh` configure l'environnement mobile en générant un fichier `.env` contenant l'adresse IP locale de votre machine, essentielle pour que l'application mobile puisse communiquer avec le backend.
- Il vous sera demandé votre IP locale, que vous pouvez obtenir avec :

```
ipconfig # sous Windows
ifconfig # sous macOS/Linux
```

Exemple d'IP locale : 192.168.1.17

Note : Si le script ne fonctionne pas, vous pouvez configurer manuellement l'application mobile en vous plaçant dans le dossier `gbz-mobile-app` puis en lançant :

```
npm install
npm start
```

5.3. Configuration. Env

Le fichier `.env` est essentiel car il contient les variables d'environnement nécessaires au bon fonctionnement des services, notamment l'URL du backend pour l'application mobile.

Le script `start-mobile.sh` génère ce fichier automatiquement avec la bonne IP locale, mais vous pouvez aussi le créer manuellement en ajoutant par exemple :

```
EXPO_PUBLIC_API_URL=http://192.168.1.17:3000
```

5.4. Lancement Docker

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Pour démarrer tous les services du projet, Docker Compose est utilisé, permettant de lancer simultanément :

- La base de données PostgreSQL
- Le backend NestJS
- L'interface d'administration React
- L'application mobile Expo

Pour lancer l'ensemble, tapez la commande suivante dans votre terminal :

```
docker compose up --build
```

Une fois les conteneurs démarrés, vous pourrez accéder à :

- PostgreSQL (service interne, sans URL d'accès directe)
- Backend sur <http://localhost:3000>
- Interface admin sur <http://localhost:5173>
- Application mobile via le script comme précisé précédemment.

5.5. Lancement Projet

Une fois les services lancés, vous pouvez :

- Ouvrir l'interface admin dans votre navigateur à l'adresse <http://localhost:5173>
- Scanner le QR Code Expo affiché dans le terminal pour lancer l'application mobile via Expo Go sur votre téléphone
- Tester l'API backend en local sur le port 3000

Cela vous permettra d'explorer l'ensemble de l'application complète, avec backend, administration et mobile fonctionnels.

6. Backend

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

6.1. Structure

Le backend de l'application est développé avec **NestJS**, un framework Node.js moderne, modulaire et très structurant. Le code est organisé en modules, chaque module correspondant à une fonctionnalité ou un domaine métier spécifique, comme par exemple la gestion des utilisateurs, des exercices ou des séances.

```

→ backend git:(main) X tree -L 2
├── Dockerfile
├── README.md
├── docker-compose.yml
├── eslint.config.mjs
├── nest-cli.json
├── package-lock.json
├── package.json
├── prisma
│   ├── schema.prisma
│   └── seed.ts
├── src
│   ├── app.controller.spec.ts
│   ├── app.controller.ts
│   ├── app.module.ts
│   ├── app.service.ts
│   ├── auth
│   ├── exercises
│   ├── main.ts
│   ├── muscles
│   ├── prisma
│   ├── prisma-client-exception
│   ├── programs
│   ├── sessions
│   ├── user-exercise-sessions
│   └── users
├── test
│   ├── app.e2e-spec.ts
│   ├── jest-e2e.json
│   ├── tsconfig.build.json
│   └── tsconfig.json
└── 12 directories, 18 files
→ backend git:(main) X

```

Cette architecture modulaire facilite grandement la maintenance et l'évolution du projet. Chaque module regroupe ses propres contrôleurs, qui gèrent les requêtes entrantes, ses services, responsables de la logique métier, ainsi que ses entités liées à la base de données.

Tout au long du développement, je me suis énormément appuyé sur **Swagger**, intégré à NestJS, qui m'a permis de documenter automatiquement l'API REST. Cet outil a été indispensable pour tester les endpoints, comprendre rapidement leur fonctionnement et assurer une communication claire entre le backend et les autres parties du projet, notamment l'application mobile et l'interface d'administration.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

La sécurité a été un aspect central du développement du backend, et plusieurs mesures ont été mises en place pour garantir la protection des données et des accès :

- **Authentification JWT avec Passport** : Pour gérer l'authentification, j'ai utilisé le module **Passport** intégré à NestJS, qui facilite la mise en place de stratégies d'authentification. La stratégie principale utilisée est celle des tokens JWT. Les utilisateurs doivent s'authentifier en obtenant un token JWT, qui est ensuite vérifié à chaque requête vers les routes protégées. Cette approche assure un contrôle d'accès fiable et performant.
- **Gestion des rôles et permissions** : Certaines routes sont réservées aux administrateurs ou aux utilisateurs authentifiés, selon leur rôle. Cette gestion fine des permissions est implémentée via des guards NestJS, permettant de filtrer l'accès en fonction des droits.
- **Validation des données** : Toutes les données reçues par l'API sont validées grâce aux DTOs (Data Transfer Objects) et aux pipes de validation fournis par NestJS. Cela garantit que les entrées respectent bien les formats attendus et évite les erreurs ou failles potentielles.
- **Protection contre les injections** : L'utilisation de Prisma comme ORM permet de bénéficier d'une protection efficace contre les injections SQL, car les requêtes sont générées de manière sécurisée.
- **Hashage des mots de passe** : Les mots de passe des utilisateurs sont systématiquement hashés avant d'être stockés en base de données, et ne sont jamais stockés ou affichés en clair, ni dans le code ni dans les logs.

La documentation officielle de NestJS a été une ressource précieuse tout au long du développement, notamment pour la mise en place sécurisée de l'authentification avec Passport et les bonnes pratiques liées à la validation et la protection des routes.

6.4. Liste Endpoints

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

Ressource	Méthode HTTP	Endpoint	Description
USERS	POST	/users	Créer un nouvel utilisateur
USERS	GET	/users	Récupérer la liste des utilisateurs
USERS	GET	/user/{id}	Récupère un utilisateur en fonction de son id
USERS	PATCH	/user/{id}	Modifie un utilisateur en fonction de son id
USERS	DELETE	/user/{id}	Supprime un utilisateur selon son id
AUTH	POST	/auth/login	Permet de se login en tant qu'utilisateur
AUTH	GET	/auth/me	Permet d'obtenir ses informations
EXERCICES	POST	/exercices	Créer un nouvel exercice
EXERCICES	GET	/exercices	Récupère tous les exercices
EXERCICES	GET	/exercices/{id}	Récupère un exercice en fonction de son id
EXERCICES	PATCH	/exercices/{id}	Modifie un exercice en fonction de son id
EXERCICES	DELETE	/exercices/{id}	Supprime un exercice en fonction de son id
MUSCLES	POST	/muscles	Créer un nouveau muscle
MUSCLES	GET	/muscles	Récupère tous les muscles
MUSCLES	GET	/muscles/{id}	Récupère un muscle en fonction de son id
MUSCLES	PATCH	/muscles/{id}	Modifie un muscle en fonction de son id
MUSCLES	DELETE	/muscles/{id}	Supprime un muscle en fonction de son id

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

SESSIONS	POST	/sessions	Créer une nouvelle séance
SESSIONS	GET	/sessions	Récupère toutes les séances
SESSIONS	GET	/sessions/{id}	Récupère une séance en fonction de son id
SESSIONS	PATCH	/sessions/{id}	Modifie une séance en fonction de son id
SESSIONS	DELETE	/sessions/{id}	Supprime une séance en fonction de son id
PROGRAMS	POST	/programs	Créer un nouveau programme
PROGRAMS	GET	/programs	Récupère tous les programmes existants
PROGRAMS	GET	/programs/{id}	Récupère un programme en fonction de son id
PROGRAMS	PATCH	/programs/{id}	Modifie un programme en fonction de son id
PROGRAMS	DELETE	/programs/{id}	Supprime un programme en fonction de son id
USERS-EXERCICE-SESSIONS	POST	/users-exercise-sessions	Créer une nouvelle liaison entre users et séances
USERS-EXERCICE-SESSIONS	GET	/users-exercise-sessions/{id}	Récupère toutes les liaisons
USERS-EXERCICE-SESSIONS	PATCH	/users-exercise-sessions/{id}	Permet de modifier une liaison en fonction de son id
USERS-EXERCICE-SESSIONS	DELETE	/users-exercise-sessions/users/{id}	Permet de supprimer une liaison en fonction de son id

7. Admin

7.1. Structure du projet

L'interface d'administration a été développée avec **Refine**, un framework React open-source, spécialement conçu pour créer rapidement des dashboards performants et modulaires. Le projet est organisé de manière claire pour faciliter la maintenance et l'évolution.

Voici la structure principale du projet **auth-antd** :

```
→ auth-antd git:(main) X tree -L 3
.
├── Dockerfile
├── README.md
├── index.html
├── package-lock.json
├── package.json
├── public
│   ├── favicon.ico
│   └── manifest.json
├── src
│   ├── App.tsx
│   ├── index.tsx
│   └── pages
│       ├── Register.tsx
│       ├── dashboard.tsx
│       ├── exercices
│       ├── muscles
│       ├── programs
│       ├── sessions
│       ├── users
│       └── providers
│           ├── AuthProvider.ts
│           ├── auth-provider.ts
│           └── data-provider.ts
├── vite-env.d.ts
├── tsconfig.json
├── tsconfig.node.json
└── vite.config.ts

9 directories, 18 files
→ auth-antd git:(main) X
```

- Le dossier **pages/** regroupe toutes les pages liées à la gestion des différents modules métiers (utilisateurs, exercices, muscles, programmes, séances).
- Le dossier **providers/** centralise la communication avec le backend, ainsi que la gestion de l'authentification et des données.
- Les fichiers de configuration et ressources statiques sont organisés à la racine et dans **public/**.

Le projet intègre une gestion stricte des accès, l'interface admin étant uniquement accessible aux administrateurs authentifiés. Cette authentification repose sur l'utilisation de tokens JWT transmis

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

au backend via le provider d'authentification Refine, ce qui assure une sécurité robuste tout au long de la navigation.

Cette organisation modulaire et sécurisée facilite le développement de nouvelles fonctionnalités tout en garantissant un contrôle précis des accès et des données.

7.2. Gestion des utilisateurs

La gestion des utilisateurs constitue une fonctionnalité clé de l'interface d'administration. Elle permet aux administrateurs de superviser et de contrôler les comptes des membres de l'application, avec plusieurs opérations disponibles :

- **Visualisation** : Affichage de la liste complète des utilisateurs inscrits, avec des informations essentielles telles que le nom, l'adresse email, le rôle, et le statut du compte.
- **Création** : Ajout manuel de nouveaux utilisateurs par les administrateurs, en renseignant les informations nécessaires.
- **Modification** : Possibilité de modifier les données des utilisateurs existants, comme leurs rôles, leur statut (actif/inactif), ou d'autres détails de profil.
- **Suppression** : Suppression sécurisée d'utilisateurs, avec une confirmation préalable pour éviter toute erreur.
- **Gestion des rôles** : Attribution et modification des rôles des utilisateurs (par exemple, administrateur ou simple utilisateur) afin de contrôler leurs droits d'accès dans l'application.

Toutes ces actions sont réalisées via une interface simple et intuitive, grâce aux composants fournis par Refine et Ant Design. La communication avec le backend se fait via des appels API REST sécurisés, garantissant la cohérence et la sécurité des opérations.

Enfin, chaque modification est immédiatement répercutée dans la base de données, grâce à Prisma, ce qui assure une synchronisation en temps réel entre l'interface d'administration et le backend.

7.3. Connexion à l'API

L'interface d'administration communique directement avec le backend via une API REST sécurisée. Cette connexion est essentielle pour récupérer, créer, modifier ou supprimer des données liées aux utilisateurs, exercices, programmes, et autres entités du projet.

Pour assurer cette communication, plusieurs éléments ont été mis en place :

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

- **Configuration de l'URL API** : L'interface admin utilise une URL d'API configurable, définie dans un fichier .env ou un fichier de configuration spécifique. Cela permet de changer facilement l'environnement (local, staging, production) sans modifier le code.
- **Gestion de l'authentification** : L'accès à l'API est protégé par un système d'authentification basé sur des tokens JWT. Lorsqu'un administrateur se connecte à l'interface, il reçoit un token qui sera attaché à chaque requête afin d'authentifier et d'autoriser l'accès aux ressources.
- **Gestion des erreurs** : Les erreurs de communication ou d'authentification sont gérées proprement dans l'interface, avec des messages clairs affichés à l'utilisateur pour l'informer en cas de problème.
- **Utilisation de Refine Data Provider** : Refine facilite la connexion à l'API via un système de "data provider" qui standardise les appels HTTP et la gestion des données. Le data provider est configuré pour interagir avec les endpoints REST du backend NestJS, en gérant automatiquement les requêtes GET, POST, PUT, DELETE.
- **Sécurité des données** : Toutes les communications sont effectuées via HTTPS en production, garantissant la confidentialité et l'intégrité des échanges entre l'interface admin et le serveur.

Cette architecture garantit une interaction fluide et sécurisée entre l'interface d'administration et le backend, assurant ainsi une gestion efficace et sécurisée des données du projet.

8. Mobile

L'application mobile GBZ a été pensée pour offrir une expérience utilisateur complète, intuitive et fluide, permettant aux passionnés de musculation de suivre facilement leurs entraînements et de gérer leurs programmes depuis leur smartphone. Le développement a été réalisé avec React Native, en s'appuyant sur Expo Go afin d'accélérer le cycle de développement et faciliter les tests sur différents appareils.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

8.1. Navigation

La navigation dans l'application mobile GBZ a été conçue pour être fluide, intuitive et accessible, afin d'accompagner l'utilisateur depuis sa première ouverture jusqu'à l'utilisation quotidienne.

Au lancement, l'utilisateur est accueilli par une **page de connexion / inscription**. Cette étape est obligatoire pour accéder aux fonctionnalités, garantissant ainsi la sécurité des données et un suivi personnalisé.

Une fois authentifié, l'utilisateur accède à la **barre de navigation principale** (navbar), située en bas de l'écran, qui permet de naviguer facilement entre les sections clés de l'application :

- **Accueil** : Un tableau de bord personnalisé présentant un résumé des prochaines séances et statistiques d'entraînement.
- **Programmes** : Liste des programmes d'entraînement disponibles ou en cours, avec possibilité de consulter les détails de chaque programme.
- **Séances** : Interface dédiée au suivi des séances en cours, avec enregistrement des séries, répétitions, poids et commentaires.
- **Exercices** : Catalogue complet des exercices classés par groupe musculaire, avec fiches détaillées.
- **Profil** : Gestion des informations personnelles, paramètres et déconnexion.

Cette organisation via une barre de navigation permanente permet à l'utilisateur de changer rapidement de section, sans perdre le contexte. La navigation au sein de chaque section est gérée via une navigation en pile (stack), ce qui permet d'ouvrir des écrans détaillés sans quitter la section principale.

Le flux utilisateur est ainsi simple et logique : il commence par s'authentifier, puis navigue entre les principales fonctionnalités via la navbar, avec une gestion cohérente de l'historique et des transitions.

8.2. Ecrans et Fonctionnalités

L'application mobile GBZ est composée de plusieurs écrans clés, chacun conçu pour offrir une expérience utilisateur optimale et répondre aux besoins spécifiques des passionnés de musculation.

Écran Connexion / Inscription

- Permet à l'utilisateur de se connecter avec ses identifiants ou de créer un nouveau compte.
- Validation des champs et gestion des erreurs pour garantir une saisie correcte.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

- Redirection automatique vers l'écran d'accueil après authentification réussie.

Écran Accueil

- Présente un résumé des prochaines séances programmées.
- Offre un accès rapide aux sections principales via la navbar.

Écran Programmes

- Liste des programmes d'entraînement disponibles ou ceux suivis par l'utilisateur.
- Possibilité de consulter le détail d'un programme : description, durée, objectifs, exercices inclus.

Écran Séances

Affiche les exercices à réaliser, avec les séries, répétitions, poids recommandés.

- Permet de saisir ses performances (répétitions, charge).

Écran Exercices

- Catalogue organisé par groupes musculaires (pectoraux, jambes, dos, etc.).
- Fiches détaillées pour chaque exercice, incluant description.

Écran Profil

- Affichage des informations personnelles et des paramètres de l'application.
- Option pour se déconnecter proprement.

Chaque écran est pensé pour être ergonomique, rapide à charger et facile à utiliser, que ce soit sur smartphone Android ou iOS. L'interface respecte une charte graphique cohérente avec l'identité visuelle du projet GBZ.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

9. Développement et Organisation

9.1. Méthodologie

Pour mener à bien ce projet, j'ai choisi une approche méthodique basée sur les bonnes pratiques du développement logiciel, tout en m'adaptant à un contexte individuel.

Le travail s'est organisé autour de plusieurs phases :

- **Analyse et conception** : définition des besoins, architecture générale, choix technologiques.
- **Développement** : implémentation progressive du backend, de l'interface admin, puis de l'application mobile.
- **Tests** : validation fonctionnelle et technique à chaque étape, notamment avec des tests unitaires et manuels.
- **Documentation** : mise à jour régulière de la documentation pour assurer une bonne traçabilité et faciliter la maintenance.

J'ai aussi beaucoup utilisé des outils de gestion de tâches (GitHub Projects) pour planifier, suivre et organiser les développements. Cela m'a aidé à rester concentré et à garder une vision claire de l'avancement global.

9.2. Gestion GIT

Le contrôle de version a été un élément clé tout au long du projet. J'ai utilisé **Git** pour gérer le code source, avec l'ensemble des dépôts hébergés sur **GitHub**.

Pour garantir une bonne qualité de gestion des versions, j'ai suivi les bonnes pratiques suivantes :

- **Branches dédiées** : développement réalisé sur des branches spécifiques à chaque fonctionnalité ou correction, avec des merges uniquement une fois les modifications validées.
- **Convention de commits** : j'ai appliqué la règle des conventional commits afin d'assurer des messages clairs, structurés et compréhensibles, facilitant la lecture de l'historique et l'identification des changements.
- **Pull requests** : même en travaillant seul, j'ai utilisé des pull requests pour relire et valider mon propre code avant de fusionner, ce qui m'a aidé à garder un code propre et cohérent.
- **Suivi des versions** : chaque version majeure ou étape importante a été taguée dans Git, ce qui facilite le retour en arrière si nécessaire.

L'utilisation de GitHub Projects, combinée à ces pratiques, m'a permis d'avoir une organisation rigoureuse et d'assurer un développement structuré et maîtrisé.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

10. Déploiement

10.1. Local

Pour le développement et les tests locaux, j'ai utilisé **Docker** et **Docker Compose** afin d'orchestrer l'ensemble des services nécessaires à l'application (backend, base de données, interface admin).

Cette approche m'a permis d'avoir un environnement de développement cohérent, identique pour tous les services, et facilement reproductible.

Le lancement local s'effectue via une simple commande : `docker compose up --build`

Cela construit et démarre tous les conteneurs, rendant l'application accessible en local sur les ports configurés.

Les scripts dédiés facilitent aussi la configuration des variables d'environnement, garantissant une mise en route rapide et sans erreur.

10.2. Déploiement en Ligne

En raison du temps limité consacré au projet, je n'ai pas encore pu effectuer de déploiement en production en ligne.

Mon intention est d'héberger l'application sur un **Raspberry Pi** personnel, ce qui permettrait de garder un contrôle total sur l'infrastructure tout en disposant d'un serveur dédié à domicile. Cette solution offre un bon compromis entre autonomie, coût réduit et accès permanent, tout en restant compatible avec la stack Docker utilisée.

Par ailleurs, j'aurais également souhaité explorer l'option de déploiement via **GitLab**, notamment pour bénéficier de ses fonctionnalités CI/CD intégrées, qui faciliteraient l'automatisation des tests, des builds et des mises en production.

Enfin, la mise en place d'un pipeline CI/CD reste un objectif à moyen terme pour garantir la maintenance et les mises à jour continues de l'application.

	Documentation Technique GBZ	Réf. Projet	PROJET-FIL-ROUGE
		Version	V1.0
		Date	24/06/2025
		Réalisation	Garrigues Hugo

11. Conclusion

11.1. Bilan personnel

Ce projet GBZ a représenté un véritable défi personnel et professionnel. Il m'a offert l'opportunité de mettre en pratique les connaissances théoriques acquises durant ma formation et de me confronter à des problématiques concrètes de développement fullstack. Travailler sur un projet complet, mêlant backend, administration et application mobile, m'a permis d'acquérir une vision d'ensemble essentielle pour tout développeur aujourd'hui. J'ai particulièrement apprécié la liberté laissée dans le choix des technologies, ce qui m'a permis d'explorer des outils qui m'attiraient depuis longtemps, comme Prisma ou Refine. Cette expérience m'a aussi permis de mieux comprendre l'importance d'une bonne organisation, d'une documentation claire et d'une gestion rigoureuse du code. Globalement, ce projet a renforcé ma confiance en mes capacités à mener à bien un projet complexe de bout en bout.

11.2. Points d'amélioration

Malgré les nombreux points positifs, plusieurs aspects mériteraient d'être approfondis ou retravaillés pour améliorer la qualité globale du projet. Tout d'abord, le déploiement en production n'a pas pu être finalisé, principalement par manque de temps. Cela limite l'accès à l'application en conditions réelles, bien que l'architecture soit déjà prête à être hébergée (notamment sur un Raspberry Pi, comme prévu initialement).

Ensuite, des améliorations notables pourraient être apportées à la base de données, tant sur la normalisation des tables que sur l'optimisation des relations et des index. Une meilleure modélisation permettrait d'anticiper la montée en charge et de garantir la scalabilité à long terme.

Du côté de l'application mobile, une attention particulière pourrait être portée à l'optimisation de l'expérience utilisateur (UX) et à l'amélioration de l'interface (UI). Certaines vues gagneraient à être retravaillées pour offrir une navigation plus intuitive, des transitions plus fluides et une hiérarchisation visuelle plus claire.

La documentation, bien qu'utile, pourrait également être étoffée, notamment sur certains aspects techniques comme les endpoints de l'API ou les configurations avancées. Cela faciliterait la prise en main pour un futur collaborateur ou une éventuelle ouverture du projet en open-source.